

HDOC Day-33

Menu-Driven Matrix Operations Using Separate Functions

Aim

To write a menu-driven C program using variable-length 2D integer arrays and separate functions to verify the associative properties of matrix addition and matrix multiplication.

Problem Statement

1. Read three matrices A, B and C of dimensions $m \times n$ and verify matrix addition: $(A + B) + C = A + (B + C)$.
2. Read matrices A ($m \times n$), B ($n \times p$) and C ($p \times q$) and verify matrix multiplication: $(A \times B) \times C = A \times (B \times C)$.
3. Provide an Exit option. For each operation, display the matrices obtained on both sides and state whether the associative property is verified.

Algorithm

1. Display a menu with options for matrix addition, matrix multiplication and exit.
2. For addition, input m and n, then read A, B and C. Compute $A+B$, $B+C$, then calculate $(A+B)+C$ and $A+(B+C)$. Display both results and compare corresponding elements.
3. For multiplication, input m, n, p and q. Read A($m \times n$), B($n \times p$) and C($p \times q$). Compute $A \times B$ and $B \times C$, then calculate $(A \times B) \times C$ and $A \times (B \times C)$. Display both results and compare corresponding elements.
4. Print whether the associative property is verified. Repeat until the user chooses Exit.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c], char name) {
    printf("Enter elements of matrix %c:\n", name);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void displayMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int sum[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) sum[i][j] = a[i][j] + b[i][j];
}
```

```

}

void multiplyMatrix(int r1, int c1, int c2,
    int a[r1][c1], int b[c1][c2], int result[r1][c2]) {
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < c1; k++) result[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

int main(void) {
    int choice;
    do {
        printf("\n1. Associative property of matrix addition\n");
        printf("\n2. Associative property of matrix multiplication\n");
        printf("\n3. Exit\nEnter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            int m, n;
            printf("Enter rows (m) and columns (n): ");
            scanf("%d%d", &m, &n);
            int A[m][n], B[m][n], C[m][n];
            int AB[m][n], BC[m][n], left[m][n], right[m][n];
            readMatrix(m,n,A,'A'); readMatrix(m,n,B,'B'); readMatrix(m,n,C,'C');
            addMatrix(m,n,A,B,AB); addMatrix(m,n,AB,C,left);
            addMatrix(m,n,B,C,BC); addMatrix(m,n,A,BC,right);
            printf("\n(A+B)+C =\n"); displayMatrix(m,n,left);
            printf("\nA+(B+C) =\n"); displayMatrix(m,n,right);
            printf(equalMatrix(m,n,left,right) ? "Associative property is verified.\n" : "Associative property is not verified.\n");
        } else if (choice == 2) {
            int m, n, p, q;
            printf("Enter m n p q: "); scanf("%d%d%d%d", &m,&n,&p,&q);
            int A[m][n], B[n][p], C[p][q];
            int AB[m][p], BC[n][q], left[m][q], right[m][q];
            readMatrix(m,n,A,'A'); readMatrix(n,p,B,'B'); readMatrix(p,q,C,'C');
            multiplyMatrix(m,n,p,A,B,AB); multiplyMatrix(m,p,q,AB,C,left);
            multiplyMatrix(m,n,p,B,C,BC); multiplyMatrix(m,n,q,A,BC,right);
            printf("\n(AxB)xC =\n"); displayMatrix(m,q,left);
            printf("\nAx(BxC) =\n"); displayMatrix(m,q,right);
            printf(equalMatrix(m,q,left,right) ? "Associative property is verified.\n" : "Associative property is not verified.\n");
        } else if (choice == 3) {
            printf("Exiting program.\n");
        } else {
            printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}

```

Test Data for Screenshots

Use small 2 x 2 matrices so the complete input and output are clearly visible in the terminal.

Operation	Dimensions	Matrices
Addition	$m = 2, n = 2$	A = [[1,2],[3,4]] B = [[5,6],[7,8]] C = [[9,10],[11,12]]
Multiplication	$m = n = p = q = 2$	A = [[1,2],[3,4]] B = [[5,6],[7,8]] C = [[1,0],[0,1]]

Output / Screenshot

```
(kali㉿kali)-[~/Desktop/apple]
$ gcc hdoc33.c -o hdoc33

(kali㉿kali)-[~/Desktop/apple]
$
```

Compilation

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc33

1. Associative property of matrix addition
2. Associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter rows (m) and columns (n): 2 2
Enter elements of matrix A:
1 2 3 4
Enter elements of matrix B:
5 6 7 8
Enter elements of matrix C:
9 10 11 12

(A+B)+C =
15    18
21    24

A+(B+C) =
15    18
21    24
Associative property is verified.
```

Matrix Addition Output

```

1. Associative property of matrix addition
2. Associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m n p q: 2 2 2 2
Enter elements of matrix A:
1 2 3 4
Enter elements of matrix B:
5 6 7 8
Enter elements of matrix C:
1 0 0 1

(AxB)xC =
19    22
43    50

Ax(BxC) =
19    22
43    50
Associative property is verified.

```

Matrix Multiplication Output

Note: Exit-option screenshot can be added separately if required by the instructor.

Paste screenshot here

Paste screenshot here

Paste screenshot here

Result

The menu-driven C program was implemented successfully using separate functions. The associative properties of matrix addition and matrix multiplication are verified by computing and comparing the matrices obtained on both sides of each equation.